



中山大學
SUN YAT-SEN UNIVERSITY

字符串进阶与练习

中山大学计算机学院



主讲人：潘茂林

目录

CONTENTS

01

凯撒密码

02

最长公共前缀

03

单词拼写

04

最大单词数

05

分割平衡字符串

凯撒密码

题目描述：凯撒密码通过替换字母完成加密，每个字母由字母表中其后特定位数的字母代替。例如，Julius Caesar将字母表向后移动8个字母的位置，然后用得到的新字母表（外轮）中的字母替换原消息中的每个字母。

加密示例：

key: 8

源码: STUDENT

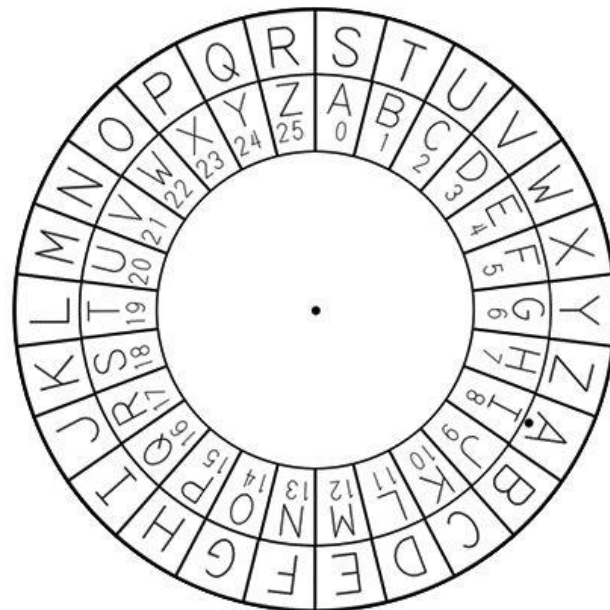
密码: ABCLMVB

解密示例：

key: -8

源码: ABCLMVB

密码: STUDENT



单击密码轮使其旋转

S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

凯撒密码

问题分解：（1）生成字母表；（2）旋转字母表；（3）完成字母加密映射

```
const char* AlphaBeta() {  
    static char s[27] = "";  
    if (s[0]) {    //检测是否已计算  
        return s;  
    }  
    else {  
        for (int i=0;i<26;i++)  
            s[i]='A'+i;  
        s[26]=0;    //字符串结束  
    }  
    return s;  
}
```

要点：

1. 如何返回**仅一次计算**的字串？
 - 使用 static 申明函数域字符串空间
 - 使用第一个字符作为是否生成标志
 - 使用 const char * 返回字符串首地址
2. 注意，字符串结束符0

凯撒密码

问题分解：（1）生成字母表；（2）**旋转字母表**；（3）完成字母加密映射

```
void AlphaShift(char str[],int k) {  
    int n = strlen(str);  
    char tmp[k+1];  
    strncpy(tmp,str+n-k,k);  
    for (int i=0;i<n-k;i++)  
        str[n-1-i] = str[n-1-k-i];  
    for (int i=0;i<k;i++)  
        str[i] = tmp[i];  
}
```

要点：

1. 本方法仅支持**顺时针旋转**外环
k 个位置
 - 字符串分隔为[0..n-k-1],[n-k..n-1]两个字串
 - 交换两个字串的位置
2. 请课后至少给出一种其他方法完成字母表旋转

凯撒密码

问题分解：（1）生成字母表；（2）旋转字母表；（3）完成字母加密映射

```
void Transform(char from[], char to[], char in_out[]) {  
    char *p = in_out;  
    for (;*p;p++) {  
        if (*p>='a' && *p<='z') {  
            *p = *p - 'a' + 'A'; //小写变大写  
            char *q = strchr(from,*p);  
            *p = *(to + (q - from));  
            *p = *p - 'A' + 'a';  
        }  
        else if (*p>='A' && *p<='Z') {  
            char *q = strchr(from,*p);  
            *p = *(to + (q - from)); //对应位置翻译  
        }  
    }  
}
```

要点：

1. 本方法依据 from 字母表与 to 字母表的对应关系，翻译 in_out 中的每个字母
 - 本案例采用指针处理字符串
 - for 语句遍历字符串
 - 大小写变换
 - 指针运算
2. 请课后给出数组操作为主的方法，完成函数

凯撒密码

优化求解：字母表编码是连续的，因此

```
void Caesar(char str[],int k,char out[]) {  
    int n = strlen(str);  
    for (int i=0; i<n; i++) {  
        if (str[i]>='a' && str[i]<='z' ) {  
            out[i]=(str[i]-'a'+ k + 26) % 26 + 'a';  
        }  
        else if (str[i]>='A' && str[i]<='Z') {  
            out[i]=(str[i]-'A'+ k + 26) % 26 + 'A';  
        }  
        else {  
            out[i] = str[i];  
        }  
    }  
    out[n]=0;  
}
```

要点：

1. 把字母映射到 0..25
2. 顺时针或逆时针移动 k 个位置
3. 再取余数完成循环移动
4. 为什么在3步之前要加 26 呢？

最长公共前缀

题目描述：编写一个函数来查找字符串数组中的最长公共前缀。
如果不存在公共前缀，返回空字符串 ""。

示例1：

数组： "flower","flow","flight"

答案： "fl"

示例2：

数组： "dog","racecar","car"

答案： ""

解释： 输入不存在公共前缀。

提示：

- $1 \leq \text{strs.length} \leq 200$
- $0 \leq \text{strs}[i].\text{length} \leq 200$
- $\text{strs}[i]$ 仅由小写英文字母组成

题目链接： [14. 最长公共前缀 - 力扣 \(Leetcode\)](#)

最长公共前缀

问题求解：按公共前缀的定义，从左至右找所有字符串都相同的前缀

```
void longestCommonPrefix(char strs[][N], size_t sz,
char pref[]){
    int p = 0;
    pref[p] = 0;    //默认前缀为空
    for (int i=0;i<N;i++) {
        char c = strs[0][i];
        if (!c) return;
        for (int j=1;j<sz;j++) {
            if (c!=strs[j][i]) return;
        }
        pref[p++] = c;
        pref[p] = 0;
    }
}
```

要点：

1. 定义、初始化字符数组
2. 描述字符数组参数
3. 初始化结果空串
4. 保证 i 小于等于长度最短的字符串长度
5. 保证结果是字符串，且 p 指向前缀结束符



例题1：单词拼写

给你一份『词汇表』（字符串数组）`words` 和一张『字母表』（字符串）`chars`。

假如你可以用 `chars` 中的『字母』（字符）拼写出 `words` 中的某个『单词』（字符串），那么我们就认为你掌握了这个单词。

注意：每次拼写（指拼写词汇表中的一个单词）时，`chars` 中的每个字母都只能用一次。

返回词汇表 `words` 中你掌握的所有单词的 长度之和。

示例 1:

输入: `words = ["cat","bt","hat","tree"], chars = "atach"`

输出: 6

解释:

可以形成字符串 "cat" 和 "hat", 所以答案是 $3 + 3 = 6$ 。

示例 2:

输入: `words = ["hello","world","leetcode"], chars = "welldonehoneyr"`

输出: 10

解释:

可以形成字符串 "hello" 和 "world", 所以答案是 $5 + 5 = 10$ 。

例题练习

以下四个题目，都给出了解题思路和初步的代码。

请完善代码，并验证函数的正确性。



例题1：单词拼写

解题思路：

- 为输入chars构造字母表，为每一个输入的字母表计数。（Hash）
- 遍历每一个word：
 - 为每一个输入word复制一份字母表：
 - 对于word中每一个字母，每遍历到一个，就对字母表中对应字母计数减1。
 - 如果计数小于0，则无法拼写，遍历下一个word
 - 如果成功遍历到word末尾，则可拼写，答案数加1



例题1：单词拼写

```
int countCharacters(char ** words, int wordsSize, char * chars){  
    short characters[26]={0};  
    short i=0,j,length=0;  
    while(chars[i]!=0){  
        characters[chars[i]-97]++;  
        i++;  
    }  
    short copied[26];  
    for(i=0;i<wordsSize;i++){  
        for(j=0;j<26;j++)  
            copied[j]=characters[j];  
        j=0;  
        while(words[i][j]!=0){  
            copied[words[i][j]-97]--;  
            if(copied[words[i][j]-97]<0) break;  
            j++;  
        }  
        if(words[i][j]!=0) continue;  
        else length=length+j;  
    }  
    return length;  
}
```

为chars建立字母表，97
为字母a的ASCII编码

遍历word的字母，对应计
数减1

遍历成功，答案数加上单
词长度



例题2：最大单词数

键盘出现了一些故障，有些字母键无法正常工作。而键盘上所有其他键都能够正常工作。

给你一个由若干单词组成的字符串 `text`，单词间由单个空格组成（不含前导和尾随空格）；另有一个字符串 `brokenLetters`，由所有已损坏的不同字母键组成，返回你可以使用此键盘完全输入的 `text` 中单词的数目。

示例 1：

输入：text = "hello world", brokenLetters = "ad"

输出：1

解释：无法输入 "world"，因为字母键 'd' 已损坏。

示例 2：

输入：text = "leet code", brokenLetters = "lt"

输出：1

解释：无法输入 "leet"，因为字母键 'l' 和 't' 已损坏。



例题2：最大单词数

解题思路：

- 从左向右遍历字符串text，遇到空格即为一个单词，总单词数加1；
- 若在单词内遍历到brokenLetters内的字母，则不能拼写的单词数加1。



例题2：最大单词数

```
int canBeTypedWords(char * text, char * brokenLetters){  
    int back = 1, words = 0, len = 0, a = 1;  
    for(len; text[len] != '\0'; len++){  
        if(text[len] == ' '){  
            a = 1;  
            back++;  
        }else{  
            if(a && strchr(brokenLetters, text[len])){  
                words++;  
                a = 0;  
            }  
        }  
    }  
    return back - words;  
}
```

遇到空格，总单词数加1

为不能拼写的单词计数

总单词数减去不能拼写的单词数为可以拼写的单词数



例题3：分割平衡字符串

在一个平衡字符串中，'L' 和 'R' 字符的数量是相同的。

给你一个平衡字符串 `s`，请你将它分割成尽可能多的平衡字符串。

注意：分割得到的每个字符串都必须是平衡字符串，且分割得到的平衡字符串是原平衡字符串的连续子串。

返回可以通过分割得到的平衡字符串的 **最大数量**。

示例 1：

输入：s = "RLRRLLRLRL"

输出：4

解释：s 可以分割为 "RL"、"RRLL"、"RL"、"RL"，每个子字符串中都包含相同数量的 'L' 和 'R'。

示例 2：

输入：s = "RLLLLRRRLR"

输出：3

解释：s 可以分割为 "RL"、"LLRRR"、"LR"，每个子字符串中都包含相同数量的 'L' 和 'R'。



例题3：分割平衡字符串

解题思路：

- 贪心策略：从左到右遍历字符串，每遇到一个平衡字符串，则答案数量加1



例题3：分割平衡字符串

```
int balancedStringSplit(char* s) {  
    int ans = 0, d = 0;  
    for (int i = 0; s[i]; i++) {  
        s[i] == 'L' ? ++d : --d;  
        if (d == 0) {  
            ++ans;  
        }  
    }  
    return ans;  
}
```

d类似一个摆针，每次回到平衡位置，则代表可以进行一个平衡字符串的分割。



例题4：第一次出现两次的字母

给你一个由小写英文字母组成的字符串 `s`，请你找出并返回第一个出现 **两次** 的字母。

注意：

- 如果 `a` 的 **第二次** 出现比 `b` 的 **第二次** 出现在字符串中的位置更靠前，则认为字母 `a` 在字母 `b` 之前出现两次。
- `s` 包含至少一个出现两次的字母。

示例 1：

输入：s = "abccbaacz"

输出："c"

解释：

字母 'a' 在下标 0、5 和 6 处出现。

字母 'b' 在下标 1 和 4 处出现。

字母 'c' 在下标 2、3 和 7 处出现。

字母 'z' 在下标 8 处出现。

字母 'c' 是第一个出现两次的字母，因为在所有字母中，'c' 第二次出现的下标是最小的。



例题4：第一次出现两次的字母

解题思路：

- 与例题1的思路一致，为字母建立计数表；
- 循环遍历字母表，为每个字母计数，等于2时退出循环，输出结果。



例题4：第一次出现两次的字母

```
char repeatedCharacter(char * s){  
    int i;  
    char alphabet[26]={0};  
  
    for(i=0;i<strlen(s);i++){  
        alphabet[s[i]-97]++;  
  
        if(alphabet[s[i]-97]==2){  
            break;  
        }  
    }  
  
    return s[i];  
}
```

为每一个字母计数，计数等于2时退出循环，输出结果



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



编制人：课题组